Lab 06 - Support approval workflow

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Prepare a refund response and demonstrate approval-bound state transitions.

Alignment: K1, K12, B08, B13-B15. Scheduled hands-on time: 45 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- orders.csv
- policies.csv
- requests.csv

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Inspect requests.csv and find R-01, R-02 and R-03. R-01 has a receipt, R-02 is missing evidence, and R-03 repeats R-01.

2. Run python3 run.py. Read the local ledger and confirm the duplicate is represented without a second simulated execution.

3. Use prompt 1 to draft the customer responses from the supplied records. Require a missing-receipt request for R-02 and no promise of payment.

4. For R-01, prepare an approval payload including order_id, amount and policy_id. Compare the exact fields a reviewer would approve.

5. Use prompt 2 to change the amount after approval. Explain why the original approval no longer applies and why the execution service must recheck it.

6. Save the draft, approval payload and duplicate-handling explanation. All execution in this lab is a local simulation; no payment or email is sent.

Acceptance checks

- Missing receipt does not become an approved refund.
- The repeated request does not create a second ledger operation.
- Approval is tied to the exact action payload.

Run python verify.py after run.py (use python3 if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder’s requirements.txt.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.

- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.